RigAssist AI

System Design Specification

Macauley Wain – Interface Designer
Jordy Gastelum – Lead Architect
Leonel Akoro – Integration Lead

University of Maryland Global Campus
CMSC 495 7303
Computer Science Capstone
Dr. Shaalan
November 6, 2026

System Design Specification

RigAssist AI is an academic mobile prototype that combines a deterministic rigging calculator, a local approved-reference library, and a constrained AI assistant. The revised architecture preserves the original safety boundary while explicitly adding the customer-data integration, human-agent escalation, supervised improvement loop, and independently scalable network services required by the Unit 3 scenario.

Team Roles and Contributions

Jordy Gastelum – Lead Architect: Owns the overall architecture, system diagrams, deterministic calculation rules, and Calculation Engine boundaries.
Macauley Wain – Interface Designer: Owns API specifications, component interfaces, data contracts, validation requirements, failure responses, and SwiftUI interaction boundaries.
Leonel Akoro – Integration Lead: Owns the AI integration strategy, Reference Service integration, approved-context flow, external LLM boundary, fallback behavior, and integration planning.

Design Requirements

- The deterministic calculator and approved local references must remain available without AI or network connectivity.
- AI may explain supported concepts using approved context but must never perform or override safety-critical calculations.
- The platform must integrate with an existing customer database through a separate, read-only Customer Data Adapter.
- Complex questions must support explicit escalation to a qualified human agent with status returned to the user.
- Interaction outcomes may inform supervised improvements, but no automatic retraining or calculation-rule changes are permitted.
- Network-facing services must be independently scalable for the conceptual requirement of approximately 10,000 concurrent users.
- Interfaces must be explicit enough for junior developers to implement and maintain without hidden dependencies.

Deliverable 1: System Architecture Diagram

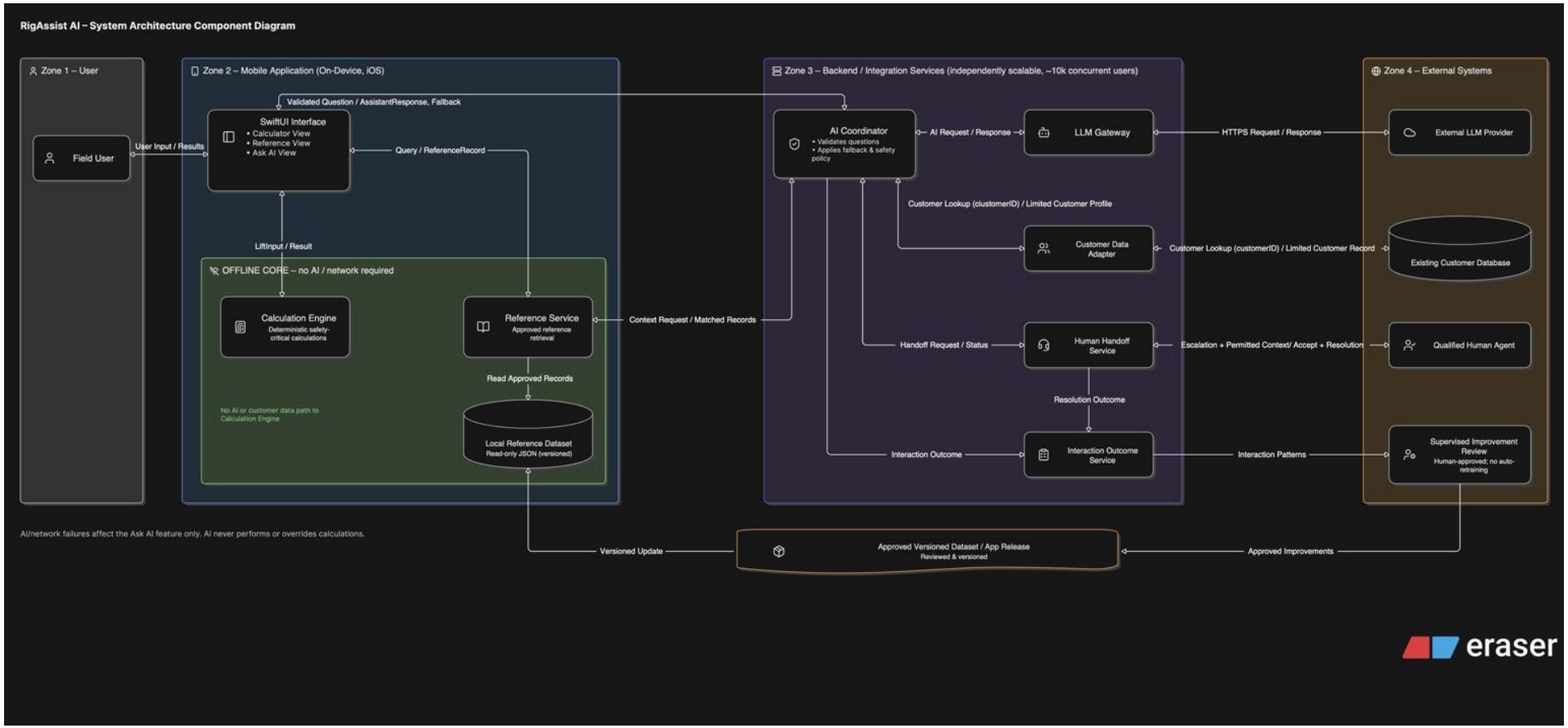


Figure 1. RigAssist AI high-level component architecture and data flow. Created by Group 3 using Eraser.

Deliverable 2: Interface Contracts

Internal on-device boundaries are expressed as Swift protocols and typed models. Network-facing boundaries use explicit request/response contracts. The contracts below define valid inputs, outputs, failure behavior, and component ownership so a developer does not need to infer implementation rules.

2.1 Calculation Engine Contract

```
func calculate(input: LiftInput) -> Result<CalculationResult, CalculationError>
```

Component	CalculationEngineProtocol / deterministic Calculation Engine
Purpose	Perform safety-critical calculations locally with no AI, customer-data, or network dependency.
LiftInput	loadWeightLb: Double, finite and > 0; configuration: SlingConfiguration; angleDegrees: Double?; slingRatedCapacityLb: Double, finite and > 0.
Configurations	verticalHitch: angleDegrees must be nil and is not used. twoLegBridle: angleDegrees is required and must be within 30.0...90.0 degrees for the approved prototype rule set.
Output	CalculationResult containing angleMultiplier: Double; loadPerLeg: Double; tension: Double; utilization: Double; withinRatedCapacity: Bool; warnings: [String]. Load and capacity values are pounds (lb); angle is degrees.
Validation	Reject nonfinite, zero, negative, missing, unexpected, or out-of-range values. Do not guess, clamp, or silently coerce safety-critical inputs.
Failures	invalidNumber; missingAngle; unexpectedAngle; angleOutOfSupportedRange; unsupportedConfiguration. A failure returns no CalculationResult and is never delegated to AI.
UI mapping	Blocking calculator message. Preserve entered values when practical and identify the field requiring correction.

2.2 Reference Service Contract

```
func record(id: String) throws -> ReferenceRecord?
func search(_ query: String, limit: Int) throws -> [ReferenceRecord]
```

Purpose	Read only approved, bundled reference records and return traceable source metadata. It does not access customer records.
Input	record(id:): trimmed, nonempty record identifier. search: trimmed, nonempty query and limit from 1 through 5 inclusive.
ReferenceRecord	id: String; topic: String; category: Category; tags: [String]; summary: String; aiContext: String; relatedCalculation: String?; source: Source; verification: Verification.

Source	title: String; publisher: String; edition: String; section: String.
Verification	status plus optional reviewedBy and reviewedOn metadata. Only records whose verification state is approved/verified may be supplied as AI context.
Valid no-result	Unknown record ID returns nil. A valid search with no match returns an empty array.
Operational failures	Blank ID/query or limit outside 1...5 returns invalidRequest. Missing bundled file returns fileNotFound. Decoding/malformed data returns decodingFailed/referenceUnavailable. Operational failures are surfaced to the caller rather than disguised as nil/empty results.
UI mapping	No match is informational. Operational failure displays “Approved references are temporarily unavailable” and does not route the question to AI without approved context.

2.3 AI Coordinator Contract

```
func ask(_ request: AssistantRequest) async -> AssistantResponse
```

Purpose	Coordinate supported natural-language questions, approved reference context, optional read-only customer context, provider access, fallback, escalation, and outcome logging.
AssistantRequest	conversationID: String (required UUID); question: String (trimmed, 1...500 characters); customerID: String? (opaque account identifier); allowCustomerContext: Bool.
Processing	Validate question; reject calculation requests; retrieve verified ReferenceRecord matches; optionally request a limited customer profile; build approved context; call LLM Gateway only when approved context exists; validate returned source IDs.
SourceCitation	recordID: String; title: String; publisher: String; edition: String; section: String. recordID must match a verified ReferenceRecord supplied in the request context.
Answered invariant	status = answered; answer is nonempty; sources contains at least one approved SourceCitation; fallbackReason is nil; nextAction may be nil or informational.
Fallback invariant	status = fallback; no unverified provider text is displayed; fallbackReason is required; nextAction is required; sources may contain approved local references when available.
Failures	unsupportedQuestion; calculationRequest; noApprovedContext; providerUnavailable; providerTimeout; malformedProviderResponse; unverifiedSource; rateLimited; customerContextUnavailable. Each maps to a controlled fallback.
Safety rule	AI never computes, modifies, or overrides CalculationResult. Source-ID validation limits citations to approved records but does not prove every generated sentence is grounded; therefore responses remain advisory and display the prototype disclaimer.

2.4 LLM Gateway / External Provider API Contract

```
POST /v1/assistant/ask
```

Caller	Mobile Ask AI workflow / AI Coordinator boundary.

Request	conversationID: UUID string; question: String 1...500; customerContext: LimitedCustomerProfile?; approvedContext: [ApprovedContextItem] with recordID and aiContext; policyVersion: String.
Provider translation	LLM Gateway converts the internal request into the configured provider format. Provider-specific model names, credentials, and endpoint details do not leak into SwiftUI or Calculation Engine contracts.
Success 200	AssistantResponse with status, answer, approved SourceCitation array, fallbackReason?, nextAction?, and disclaimer.
External call	HTTPS request/response to the configured LLM provider. Provider output is untrusted until the AI Coordinator validates referenced record IDs and response invariants.
Timeout	10 seconds for the prototype service boundary.
HTTP mapping	400 invalidRequest; 408/504 providerTimeout; 429 rateLimited; 5xx/provider network failure providerUnavailable; malformed provider payload malformedProviderResponse.
Scaling	Requests are stateless at the service boundary so network-facing instances can scale independently for the conceptual ~10,000 concurrent-user requirement.

2.5 Customer Data Adapter Contract

```
func profile(customerID: String) async -> Result<LimitedCustomerProfile,
CustomerDataError>
```

Purpose	Translate between RigAssist AI and the existing customer system without coupling the app to the database schema.
Access mode	Read-only for this prototype. No create, update, or delete operation is exposed.
Request	customerID: trimmed, nonempty opaque identifier. The adapter sends only the identifier required for the lookup.
Response	LimitedCustomerProfile: customerID: String; displayName: String?; organizationName: String?; accountStatus: String?; supportTier: String?. No calculation rules or sling-capacity values are sourced from customer data.
Failures	invalidCustomerID; notFound; accessDenied; customerSystemUnavailable; malformedCustomerRecord.
AI behavior	Customer context is optional. Failure to retrieve it does not change deterministic calculations. If the question requires customer context, AI returns a fallback or offers human escalation.
Privacy	Only permitted fields needed for support context are returned to the AI Coordinator. Logs exclude unnecessary customer content.

2.6 Human Handoff Service Contract

```
func createHandoff(_ request: HandoffRequest) async -> Result<HandoffStatus,
HandoffError>
func status(conversationID: String) async -> Result<HandoffStatus, HandoffError>
```

Trigger	Use for a complex/unsupported question requiring qualified human assistance, or when the user explicitly requests escalation. Invalid calculator input is not an escalation trigger; it remains a local validation error.
HandoffRequest	conversationID: UUID string; escalationReason: enum {complexQuestion, unsupportedTopic, customerContextRequired, userRequested}; issueSummary: String 1...1000; permittedContext: [ContextItem]; customerID: String?; createdAt: ISO-8601 timestamp.
Permitted context	Approved reference citations, limited customer profile fields, and a concise issue summary. Do not include calculation formulas, provider credentials, or unrelated user content.
HandoffStatus	conversationID; state: waiting | accepted | resolved | unavailable; agentDisplayName: String?; message: String; updatedAt: ISO-8601 timestamp.
Failures	invalidRequest; handoffUnavailable; noAgentAvailable; accessDenied; serviceUnavailable.
UI mapping	waiting shows that assistance was requested; accepted shows agent acceptance; unavailable/noAgentAvailable gives a clear next action; resolved may display the approved resolution summary.
Outcome	Resolved handoffs emit a ResolutionOutcome to the Interaction Outcome Service for supervised review.

2.7 Interaction Outcome and Supervised Improvement Contract

```
func record(_ outcome: InteractionOutcome) async -> Result<Void, OutcomeError>
```

Purpose	Capture interaction categories and resolutions so recurring issues can be reviewed without automatic model retraining.
InteractionOutcome	conversationID: UUID; outcomeType: answered | fallback | escalated | resolved; category: String; fallbackReason: String?; approvedSourceIDs: [String]; resolutionSummary: String?; occurredAt: ISO-8601 timestamp.
Review flow	Authorized human reviewer examines aggregated patterns, recurring fallback reasons, and human resolutions. Proposed changes are checked before approval.
Allowed improvements	Approved reference content, retrieval rules, AI instructions/policy, or support workflow documentation.
Prohibited automatic changes	No automatic retraining, no direct production prompt mutation, and no modification of deterministic Calculation Engine formulas from interaction data.
Release path	Approved changes become a reviewed, versioned dataset/configuration or application release. The on-device dataset is updated only through that controlled release path.
Failures	invalidOutcome; storageUnavailable; reviewQueueUnavailable. Logging failure must not block calculator/reference use or cause AI to fabricate a result.

2.8 SwiftUI Presentation Contract

Purpose	Collect input and display calculator, reference, AI, escalation, validation, and safety states.
Allowed dependencies	CalculationEngineProtocol, ReferenceServiceProtocol, AI/Coordinator client, and typed response models.
Prohibited logic	No rigging formulas, direct JSON parsing, database access, provider credentials, raw provider requests, or AI source-validation logic in views.
Calculator flow	Build LiftInput only after local field validation; submit to Calculation Engine; display CalculationResult or mapped CalculationError.
Reference flow	Submit query/record ID to Reference Service; display approved record and source metadata; distinguish no match from operational failure.
AI flow	Submit validated question; display answered response only with approved citations; otherwise display fallback and next action; expose human escalation when appropriate.
Safety display	Clearly state that RigAssist AI is an academic prototype, AI responses are advisory, and qualified human verification is required for operational decisions.

2.9 Shared Validation and Failure Policy

- Validate at the earliest responsible boundary and never silently coerce invalid safety-critical values.
- Calculation failures are blocking and remain inside the deterministic calculator workflow; they are never delegated to AI.
- A valid “no result” is distinct from an operational service failure in Reference Service and Customer Data Adapter contracts.
- AI answers require approved context and approved citations. Unsupported, ungrounded, malformed, or unavailable responses become fallbacks.
- Customer data is read-only and cannot modify calculation rules or approved reference records.
- Complex questions may be escalated to a qualified human agent; invalid calculator input is corrected locally instead of escalated.
- AI/provider/customer/handoff failures degrade only the affected network feature. Offline calculator and local reference functions remain available.
- Logs exclude credentials and minimize user/customer content. Interaction outcomes retain only information needed for supervised review.

Deliverable 3: Design Rationale

RigAssist AI uses a modular architecture that separates presentation, deterministic calculation, approved reference retrieval, customer-data access, AI coordination, human escalation,

and supervised improvement into focused components. This separation of concerns is especially important because the application combines safety-related rigging calculations with probabilistic AI assistance. The SwiftUI interface collects input and displays results, but it does not contain calculation formulas, parse the reference dataset directly, or communicate with an external model. The Calculation Engine alone performs deterministic calculations. The Reference Service owns the approved local dataset. Network-dependent responsibilities remain outside the offline core.

High cohesion is maintained by keeping related behavior inside the component that owns it. Calculation validation and results stay with the Calculation Engine; approved reference retrieval stays with the Reference Service; customer lookups stay with the Customer Data Adapter; and escalation stays with the Human Handoff Service. Low coupling is supported through explicit protocols and typed data contracts. Components exchange defined request and response models instead of reaching into one another's internal state. This makes each module easier to test independently and gives junior developers a smaller, clearer area of responsibility when maintaining the system.

AI integration follows the same boundary discipline. The AI Coordinator validates a supported question, requests approved context, and may obtain a limited customer profile through the Customer Data Adapter. It then sends only permitted context through the LLM Gateway. Provider output is treated as untrusted until source identifiers are checked against the approved context. Complex questions can be escalated through the Human Handoff Service with a conversation identifier, escalation reason, issue summary, and permitted context. Handoff status is returned to the user rather than leaving the escalation implicit.

Interaction outcomes also create a supervised learning loop without allowing automatic retraining or formula changes. AI outcomes and human resolutions are recorded by the Interaction Outcome Service. A human reviewer can identify recurring fallback categories or support issues and approve changes to reference content, retrieval rules, or AI instructions. Approved changes reach the

application only through a reviewed, versioned dataset or application release. Deterministic calculation rules remain outside this process.

The major tradeoff is offline reliability versus network-enabled AI capability. Keeping the calculator and reference library on-device improves availability when connectivity is poor and prevents an external AI failure from disabling core functions. The cost is that local reference content must be versioned and updated through releases, while AI and customer-support features still depend on network services. The backend boundary therefore uses stateless service contracts that can scale independently for the conceptual 10,000-concurrent-user requirement.

Customer records remain read-only, preventing support interactions from altering operational data and reducing coupling in prototype. This architecture favors explicit contracts over hidden dependencies. OpenAPI's specification approach demonstrates the value of formally describing service operations, inputs, outputs, and errors so implementations can evolve while preserving agreed boundaries (OpenAPI Initiative, 2024). For RigAssist AI, those boundaries support maintainability, controlled failure behavior, independent testing, and a clear rule: AI may explain approved information, but it never performs or overrides a safety-critical calculation. Overall.

References

OpenAPI Initiative. (2024). OpenAPI specification (Version 3.1.1).

https://spec.openapis.org/oas/v3.1.1.html

Appendix A: Contract Examples

Example AI Request

```
POST /v1/assistant/ask
{
  "conversationID": "7D0F...",
  "question": "What should I review before selecting a sling configuration?",
  "customerID": "CUST-1042",
  "allowCustomerContext": true
}
```

Example Answered Response

```
{
  "status": "answered",
  "answer": "<approved-context explanation>",
  "sources": [{
    "recordID": "REF-001",
    "title": "<source title>",
    "publisher": "<publisher>",
    "edition": "<edition>",
    "section": "<section>"
  }],
  "fallbackReason": null,
  "nextAction": null,
  "disclaimer": "Academic prototype; verify operational decisions with a qualified
person."
}
```

Example Fallback Response

```
{
  "status": "fallback",
  "answer": "",
  "sources": [],
  "fallbackReason": "noApprovedContext",
  "nextAction": "Review the approved reference library or request qualified human
assistance.",
  "disclaimer": "Academic prototype; verify operational decisions with a qualified
person."
}
```